

R-Control IP4 – HTTP Legacy API (CGI)

What is this?

A very simple HTTP interface: a single call in the browser or via **curl** is enough to switch. No login, no JSON, no headers.

This interface exists for the Rutenbeck Remote app and for existing installations. For new projects prefer the [HTTP/JSON API](#) — it is secured and can do considerably more.

Important: This interface has **no authentication**. Anyone who can reach the device on the network can switch it. Only enable it on an isolated network.

Getting there in 2 minutes

Step 1: Enable it

Web interface → **Settings** → **Interfaces** → **HTTP Legacy API** → tick, save.

Step 2: Switch

Toggle output 1 — just open it in a browser:

```
http://192.168.0.3/leds.cgi?led=1
```

Every call **toggles** (off → on → off). It cannot set a fixed state; use the HTTP/JSON API for that.

Step 3: Read the state

```
http://192.168.0.3/status.xml
```

Returns a small XML document with the state of all outputs and the temperature.

Impulse — for example a door opener for 3 seconds

```
http://192.168.0.3/impl.cgi?ausg100:00:03normal
```

This reads as: output **1**, duration **00:00:03**, mode **normal** (impulse ON). Use **reset** instead of **normal** for an impulse OFF.

Examples

```
# Toggle output 2
curl "http://192.168.0.3/leds.cgi?led=2"

# Read the state
curl "http://192.168.0.3/status.xml"

# Switch output 1 on for 5 seconds
curl "http://192.168.0.3/impl.cgi?ausg100:00:05normal"
```

From here on follows the **complete reference part** with all details.

This document describes the **HTTP Legacy API** (CGI endpoints) of the R-Control IP4 from an integration perspective. It targets customers/partners who need a very slim, easily scriptable HTTP interface (e.g. for Home Assistant, PLC/SCADA, test tools).

Status: Implementation in `components/http_legacy_api` (firmware repo). The HTTP Legacy API is intentionally **minimalistic** and **not authenticated**.

1. Overview

Purpose

- Simple switching of a relay via HTTP GET (toggle).
- Trigger a timed impulse (impulse ON/OFF for a defined duration).
- Retrieve a small XML status document (relay states + NTC status/temperature).

Characteristics

- Transport: HTTP (unencrypted) or HTTPS (if enabled on the device)
- Semantics: GET requests, responses as `text/plain` or `application/xml`
- **No authentication / no authorization**

2. Activation & Configuration

The HTTP Legacy API is disabled by default and is enabled via the device configuration.

2.1 Configuration Key

- `config.interfaces.http-legacy.enabled` (bool)
 - `true` = CGI endpoints are registered
 - `false` = endpoints are not available

2.2 Port / Base URL

The legacy endpoints are served by the same HTTP server as UI/REST:

- HTTP port: `config.network.lan.eth0.http.port` (default usually `80`)
- HTTPS: `config.network.lan.eth0.https.enabled` (optional)

Important:

- `config.interfaces.http-legacy.port` is included in mDNS TXT fields, but the CGI endpoint implementation itself is attached to the existing HTTP server handle.
- In practice this means: the legacy endpoints are reachable via the device base URL:
 - `http://<device>/leds.cgi`
 - `http://<device>/impl.cgi`
 - `http://<device>/status.xml`

2.3 mDNS / Service Discovery

If mDNS is enabled, the `_http._tcp` service may contain TXT records (including `http_legacy=true/false`, `http_legacy_port=<port>`).

3. Relay Channels

The legacy HTTP API uses a **1-based** channel index:

- `led=1` corresponds to relay 1
- ...

Internally, the firmware determines the number of configured outputs via `config.peripherals.switch_outputs[]`.

For `status.xml`:

- **4 channels are always output** (`<led1>...</led1>` through `<led4>...</led4>`). If the device has fewer than 4 outputs configured, missing channels are reported as `0`.

4. Endpoints

4.1 Toggle Relay

Request

- `GET /leds.cgi?led=<n>`

Parameter

- `led` (1..N)

Response

- Body: `OK\n` or `ERR\n`
- Content-Type: `text/plain; charset=utf-8`
- Cache: `Cache-Control: no-store, no-cache, must-revalidate`

HTTP status codes

- `200 OK` on success
- `400 Bad Request` on invalid parameters
- `500 Internal Server Error` on internal errors (e.g. relay switching failure)

Example

```
curl "http://r-control-ip4.local/leds.cgi?led=2"
```

4.2 Impulse (Impulse ON/OFF with Duration)

Request

- GET /impl.cgi?ausg<channel><HH:MM:SS><normal|reset>

Meaning

- **normal** = impulse ON
- **reset** = impulse OFF

Parameter format

- Prefix: **ausg**
- <channel>: 1..N (decimal, no separator)
- <HH:MM:SS>: exactly 8 characters, e.g. **00:00:10**
 - **MM** and **SS** must be within **00..59**
 - Total time must be **> 0**
- Suffix: **normal** or **reset**

Response

- Body: **OK\n** or **ERR\n**
- Content-Type: **text/plain; charset=utf-8**

HTTP status codes

- **200 OK** on success
- **400 Bad Request** for invalid query format / invalid channel / invalid duration
- **500 Internal Server Error** for internal errors (e.g. relay controller error)

Examples

```
# Turn relay 1 on for 10 seconds (impulse ON)
curl "http://r-control-ip4.local/impl.cgi?ausg100:00:10normal"

# Turn relay 1 off for 10 seconds (impulse OFF)
curl "http://r-control-ip4.local/impl.cgi?ausg100:00:10reset"
```

Note: The query string is interpreted as a single payload (no classic **key=value**).

4.3 Status (XML)

Request

- GET /status.xml

Response

- Content-Type: application/xml; charset=utf-8
- Cache: Cache-Control: no-store, no-cache, must-revalidate
- Body (example):

```
<response>
<led1>0</led1>
<led2>1</led2>
<led3>0</led3>
<led4>0</led4>
<pot0>22.50</pot0>
</response>
```

Field description

- <led1> .. <led4>: 0 or 1
- <pot0>: temperature/status field (text or number)
 - With valid temperature: temperature in °C with 2 decimal places (e.g. 22.50)
 - With NTC out-of-range LOW: unterbrochen
 - With NTC out-of-range HIGH: kurzschluss
 - If no data is available: nicht angeschlossen

5. Error Behavior

The legacy HTTP API signals errors intentionally in a simple way:

- Response body: ERR\n
- Status code: typically 400 (client error) or 500 (server error)

5.1 Common causes for 400 Bad Request

- Missing query or query too long (internal limit: 64 characters)
- /leds.cgi:
 - led missing
 - led not numeric or out of valid range
- /impl.cgi:
 - query does not start with ausg
 - suffix is not normal/reset
 - time format is not exactly HH:MM:SS
 - duration 00:00:00

6. Integration Notes (Best Practices)

6.1 Toggle vs. Set

/leds.cgi toggles the state. For deterministic behavior in automation systems:

- Query the current state via `/status.xml` first.
- Only toggle if a state change is actually required.

If an integration strictly needs “set on/off”, the JSON REST API or the WebSocket gateway is recommended for new integrations.

6.2 Avoid Caching

Responses send `Cache-Control: no-store...`. Still, integrators should not route requests through caches/proxies that aggressively cache GET requests.

6.3 Robustness

- On `ERR\n`, also evaluate the HTTP status code (400 vs 500).
- Keep client-side timeouts/retries moderate (LAN typically 1–3 seconds timeout, 0–2 retries).

7. Security Notes

This interface is **not protected**. Any client in the network segment can switch relays.

Recommended measures:

- Use only in a **trusted LAN/OT network**
- Restrict access via firewall/VLAN to defined source IPs/hosts
- Keep HTTP Legacy API disabled (`enabled=false`) and enable only when needed
- For internet/WAN-adjacent scenarios: use the authenticated REST API + HTTPS instead

8. Examples

8.1 Example (curl)

```
# Toggle relay 2
curl -i "http://r-control-ip4.local/leds.cgi?led=2"

# Impulse 30 seconds ON on relay 1
curl -i "http://r-control-ip4.local/impl.cgi?ausg100:00:30normal"

# XML status
curl -s "http://r-control-ip4.local/status.xml"
```

8.2 Example (Home Assistant – REST Sensor Idea)

The XML response can be parsed with templates (see also `docs/API.md`).

9. Compatibility & Changes

The HTTP Legacy API is a legacy interface.

For new integrations with long-term maintenance requirements, it is recommended to use the authenticated JSON REST API or the WebSocket gateway instead (see `docs/API.md`).